Linear Regression

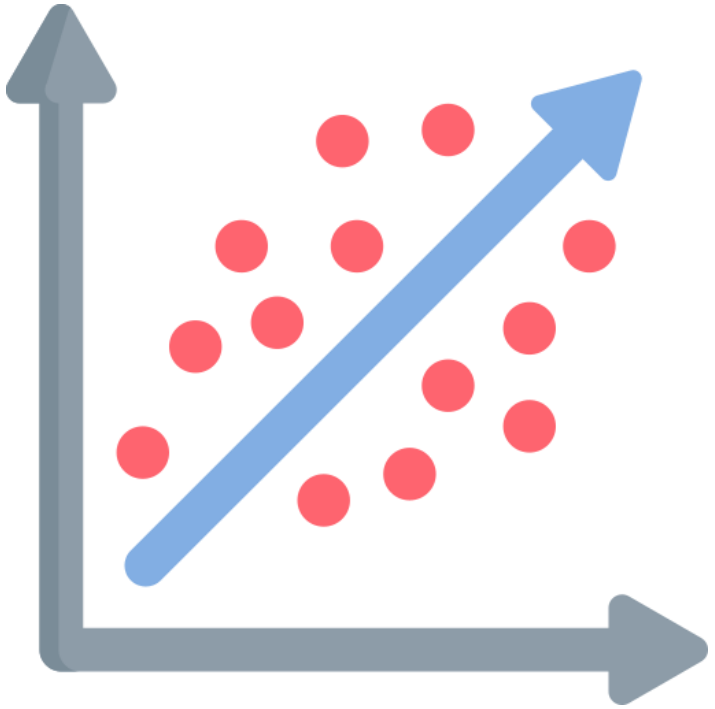


Outline

- Linear Regression with One Variable
- Multiple Linear Regression
- Preparing Data for Regression
- Regression Evaluation

Some of slides are based on the slides from
https://www.deeplearning.ai/courses/machine-learning-specialization/

Linear Regression with One Variable



ML: learn a Predictive Function that minimizes the cost

🎯 Objective

Learn a function **f(x)** that predicts the target **ŷ** from input **x** minimizing a **loss function**

0. Initialize: set the model parameters with random values

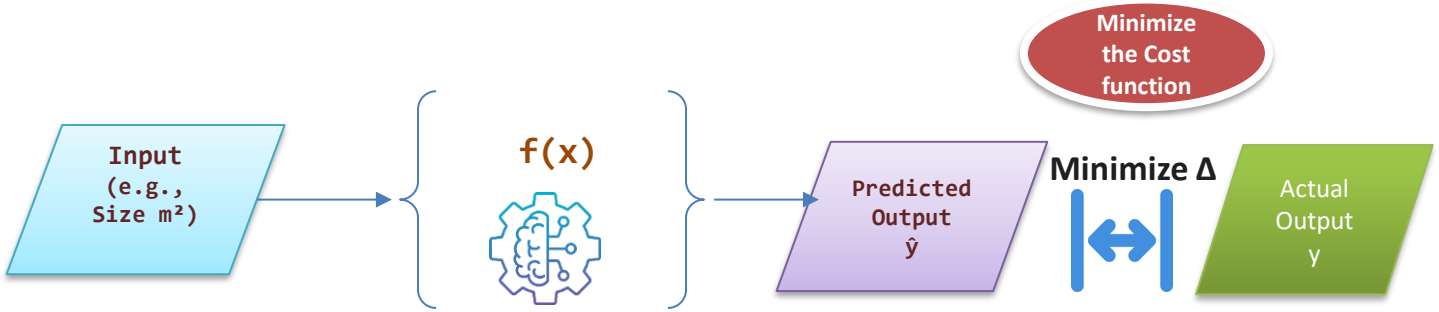
🔁 **Training Loop** (Repeat steps 1–3 until a stopping criterion is met)

1. Predict: compute ŷ = f(x) with current parameters

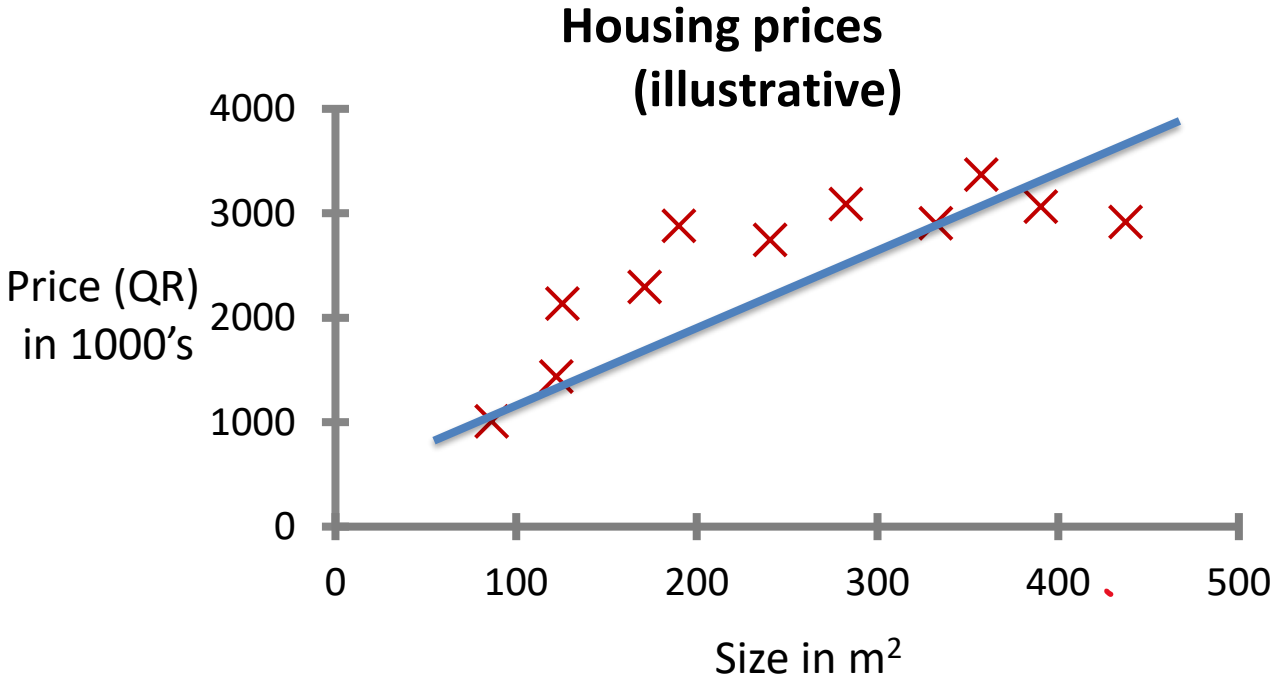
2. Evaluate: Calculate the loss by comparing **ŷ** with the actual **y**

3. Update: Adjust the model parameters to reduce the loss

- Error = Loss = difference between the Predicted Output the Actual Output



Linear Regression with One Variable



Linear regression with one variable aka **Univariate linear regression**

🏠 **Example: Housing Price Prediction**

- **Input (x):** Size of the house (m^2)
- **Output (y):** House price (in QR 1000s)

🧠 **What is Linear Regression?**

- A **supervised learning** algorithm used to predict a **continuous value**
- Predicts output **ŷ** using input **x**
- Fits a **straight line** that best represents the data

Illustrative housing prices

Size in m² (x)	Price (QR) in 1000's (y)
420	4600
280	2320
310	3150
170	1780
…	…

Notation:

m = Number of training examples

x = ***input*** variable (aka feature)

y = ***output*** variable (aka target variable)

$(x^{(i)}, y^{(i)})$ – the ith training example

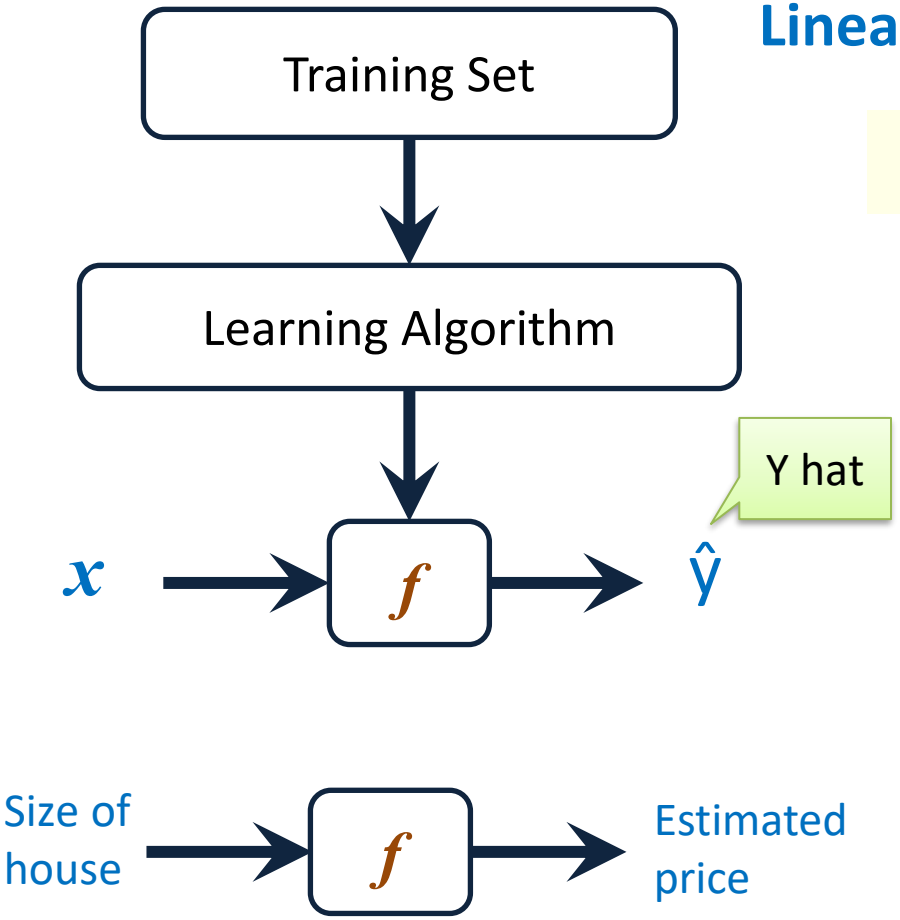
$x^{(1)} = 420$

$y^{(1)} = 4600$

$x^{(2)} = 280$

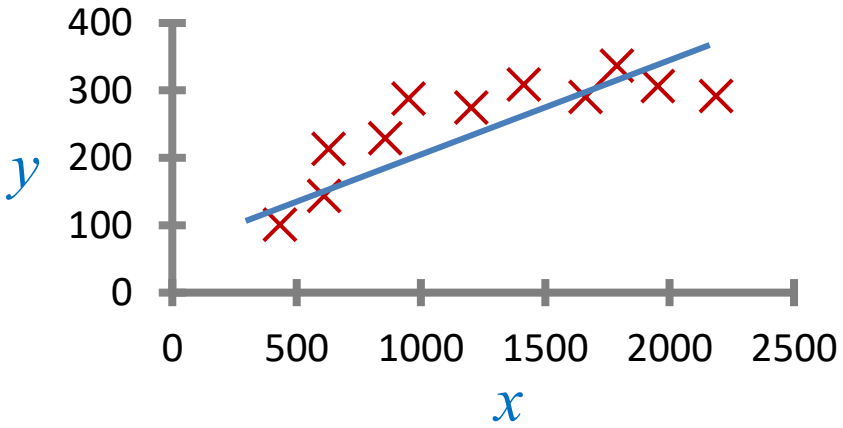
$y^{(2)} = 2320$

Linear Regression Model Representation



$$\hat{y} = f(x) = wx + b$$

w, b are parameters (coefficients) to learn from the training set



🎯 Learning Objective

- Find the **optimal values** w, b that **minimize** the Cost Function:
  - Error between predicted $\hat{y}$ and actual values y
  - **Cost function** = $\frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$ over all training examples

Mean Squared Error (MSE)

Model Representation & Parameters

$$f(x) = wx + b$$

- ***w*** is the slope of the line
- ***b*** is the y-intercept of the line

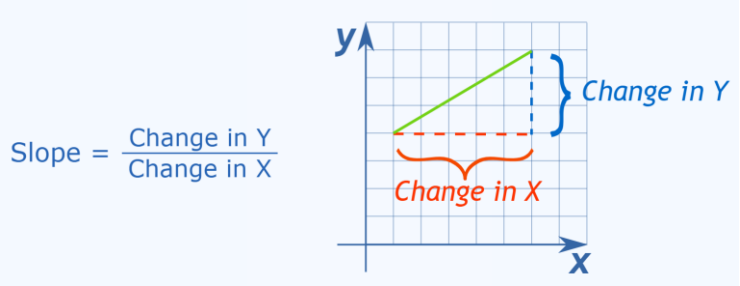
🔍 **How do w and b affect the line?**

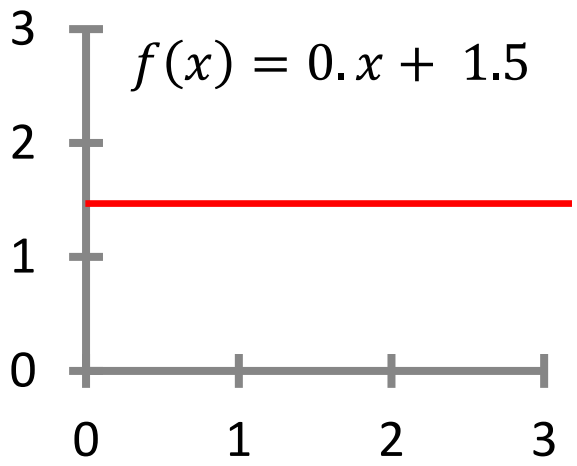
✅ **Changing b** (intercept):

- Moves the line **up or down**
- Controls the predicted value when $x = 0$

✅ **Changing w** (slope):

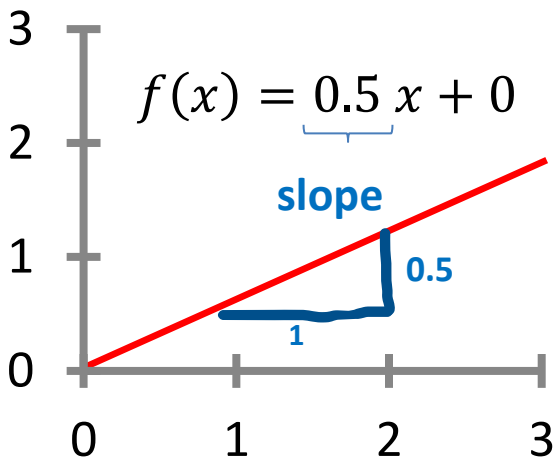
- Changes the **steepness** of the line
- Controls how fast y increases or decreases with x





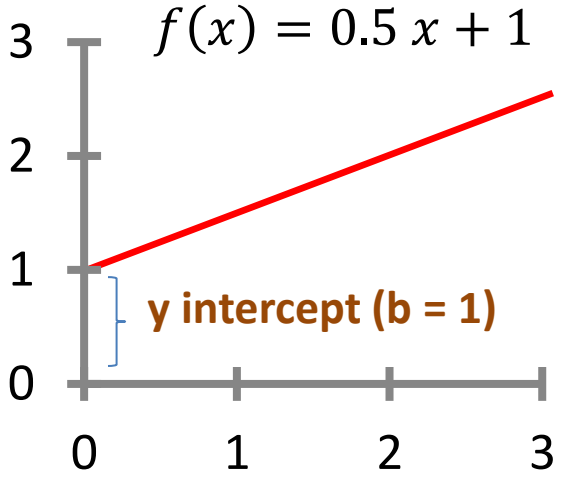
$w = 0$

$b = 1.5$



$w = 0.5$

$b = 0$



$w = 0.5$

$b = 1$



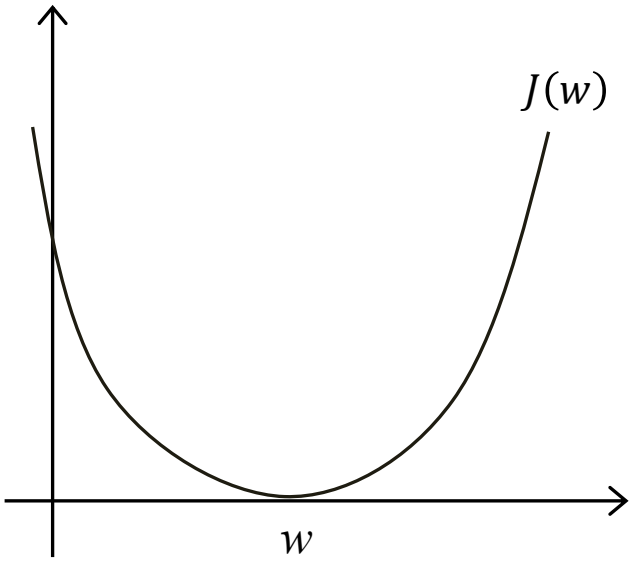
Visualizing the Cost Function $J(w)$

We set **b=0** to allow visualizing J(w) in 2D

$$With\ b=0\ \Rightarrow \boldsymbol{f(x) = wx}$$

🎯 **Learning objective:**
Find the value of w that **minimizes** $J(w)$

$$J(w) = \frac{1}{m}\sum_{i=1}^{m}(\mathrm{w}x^{(i)} - y^{(i)})^2$$

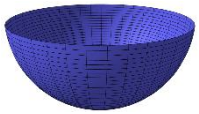


Key Observations

- Each value of w produces:
  - 📈 a different prediction line
  - 📉 a different cost value $J(w)$
- The cost function $J(w)$ has a **bowl-shaped (convex)** curve 🥣
- ✅ Guarantees **one global minimum** that minimizes $J(w)$
- The minimum of $J(w)$ can be efficiently reached by following the slope — this is the basis of Gradient Descent



`slides\regression.xlsx`



Visualizing the Cost Function $J(w,b)$

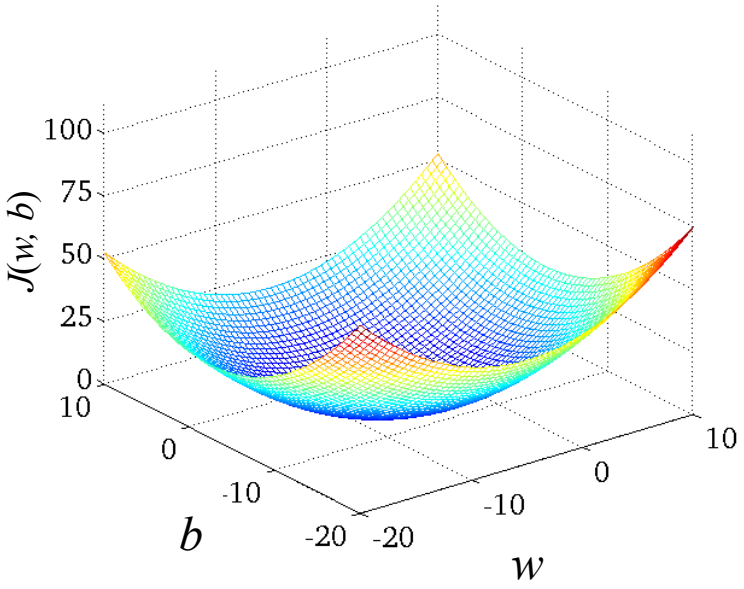
$$f(x) = wx + b$$

Linear Regression Model

$$f(x) = wx + b$$

Cost Function (MSE):

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m}(f(x^{(i)}) - y^{(i)})^2$$



Key Observations 🔍

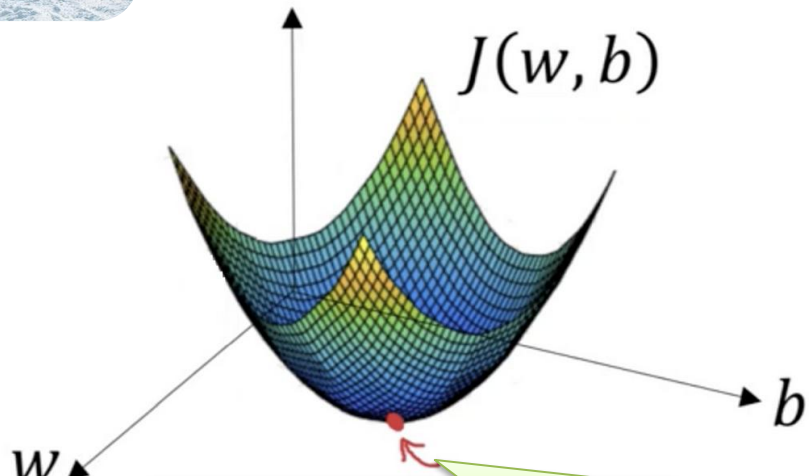
- Each pair (w, b)produces:
  - 📈 a different prediction line
  - 📊 a different cost value
- When **both** w **and** b are considered:
  - $J(w, b)$forms a **convex, bowl-shaped surface** 🎨
  - ✅ Only **one global minimum** exists
- The minimum lies at the **bottom of the bowl** ⬇️

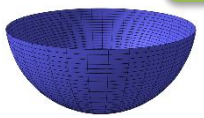
➡️ This minimum can be efficiently reached by **following the slope** of the surface → Gradient Descent



07.regression
\01_cost_function_vis







Bowl shape
Convex function

$J(w,b)$

🧠 Key Idea

Gradient Descent iteratively **moves downhill** on the cost surface to **find the optimal** w and b that minimizes that cost function

➡️ This is the core optimization method used to train linear regression models

Gradient Descent Algorithm

Goal 🎯: Learn the optimal parameters w and b that minimize the cost function.

Univariate Linear Regression Model:

$$\hat{y}^{(i)} = f\left(x^{(i)}\right) = wx^{(i)} + b$$

Cost Function (Mean Squared Error):

$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2$$

Where:

- **m** = number of training examples
- i = index of a training example

$$i = 1,2,\dots,m$$

- $x^{(i)}$ = input feature of example i
- $y^{(i)}$ = actual target value of example i
- $\hat{y}^{(i)}$ = predicted value of example i

1. **Initialize parameters with random values** (e.g., $w = 0,\ b = 0$)
2. **Compute Gradients (Partial Derivatives)** 📐

Measure how the cost changes with respect to each parameter:

$$\frac{\partial J}{\partial w} = \frac{2}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})\,x^{(i)}$$

$$\frac{\partial J}{\partial b} = \frac{2}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})$$



07.regression\
04_gradient_descent_one_var

3. **Update parameters** 🔄

Updates move parameters **downhill** ⬇️ toward lower cost

α:learning rate 🎚️

$$w := w - \alpha\frac{\partial J}{\partial w}$$

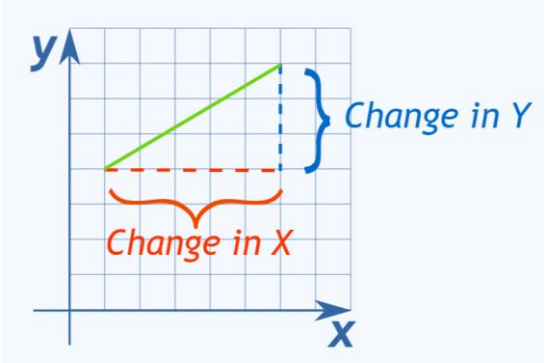
$$b := b - \alpha\frac{\partial J}{\partial b}$$

4. **Repeat** 🔁 2 and 3 until the cost stops decreasing (convergence ✅)

📐 Derivative 101

🔍 What Is a Derivative?

✅ A **derivative** tells us how fast a function changes
✅ It is simply the **slope** of the function 📈

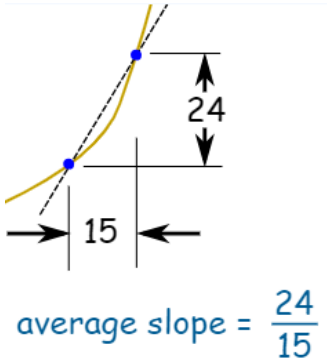


📊 Average Slope (Between Two Points)

Given two points on a curve:

$$\text{Average } slope = \frac{\Delta y}{\Delta x}$$

- Measures how much **y changes** when **x changes**
- Represents the **slope of the straight line** between two points



🧠 Why This Matters for Machine Learning

To minimize error, we need to know **which direction decreases the cost fastest**

➡️ Derivatives give us the **slope of the cost function**
➡️ Slope tells us how to **update parameters** in Gradient Descent 🚀

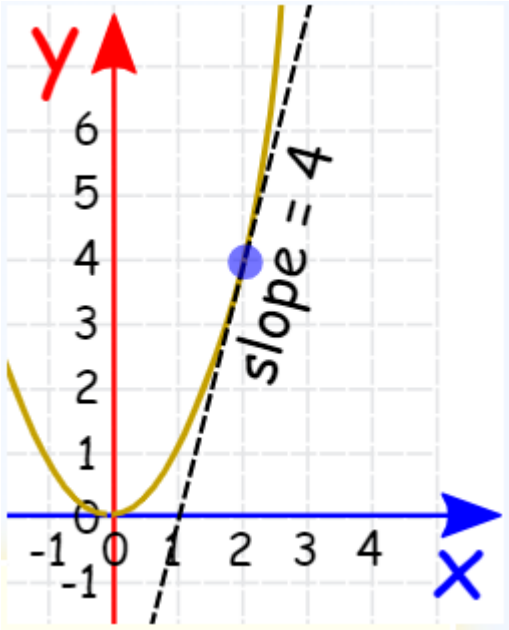
Interpretation of Derivative

- So what does $\frac{d}{dx}x^2 = 2x$ mean?
- The **derivative** of a function tells us the **slope of the curve at a given point**

👉 It answers the question:

If I move a little in x, how fast does y change?

- For function x^2, the slope at any point is **2x**
  - When **x=2** the slope is **2x = 4** → Curve rises **moderately fast** 📈
  - When **x=5** the slope is **2x = 10** → Curve rises **very steeply** 🚀





07.regression\
02_gradient_decent_x_square_animation.py

The Impact of the Learning Rate (α)

Gradient Descent Update Rule $$w \leftarrow w - \alpha \frac{dJ(w)}{dw}$$

The learning rate **α controls the step size** taken in the direction of the negative gradient.

🔹 If α is too small

- Updates to w are **very small** → $J(w)$ decreases slowly

✅ Guaranteed to move toward the minimum → slow but safe

❌ **Convergence is slow:** *takes many iterations*

🔹 If α is too large

- Updates to w are **too big** → fast but risky

❌ The algorithm may **overshoot the minimum:** $J(w)$ may oscillate or even increase

❌ **May fail to converge or diverge:** *Gradient descent becomes unstable*

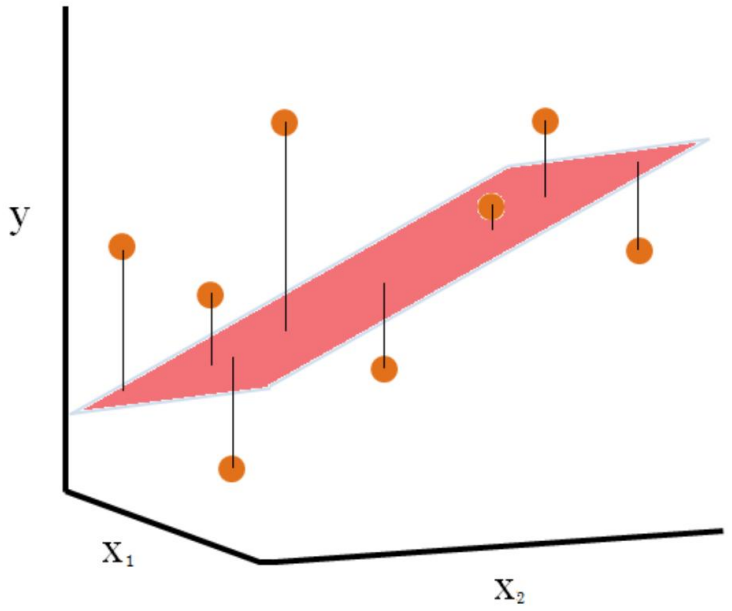
✅ Choosing α in Practice

- Start with a **moderate value** (e.g., 0.01 or 0.1)
- Select an α where $J(w)$ decreases smoothly on every iteration

👍 **Good α → fast and stable convergence**

→ If gradient descent is not working, first try reducing α before changing anything else

Multiple Linear Regression



Linear Regression with multiple variables

Illustrative data

Size (m^2)	Number of bedrooms	Number of floors	Age of home (years)	Price (QR 1000)
420	5	1	45	4600
280	3	2	40	2320
310	3	2	30	3150
170	2	1	36	1780
…	…	…	…	…

Notation:

n = number of features

x_j = j^{th} feature

$\vec{x}^{(i)}$ = features of ith training example

$x_j^{(i)}$ = value of feature j in i^{th} training example

Multiple Linear Regression:

$$f(\boldsymbol{x}) = \boldsymbol{w_1 x_1} + \boldsymbol{w_2 x_2} + \cdots + \boldsymbol{w_n x_n} + \boldsymbol{b}$$

Example

$f(x) = 0.1\,x_1 + 4\,x_2 + 10\,x_3 + -2\,x_4 + 80$

x_1: size; x_2: #bedrooms; x_3: #floors; x_4: years; 80: base price

📐 Multiple Linear Regression — Vector Form

📊 Model Representation

Given **m** training samples and **n** input features, the **vectorized linear model** is

$$\hat{\mathbf{y}} = X\mathbf{w} + b$$

Where:

$X \in \mathbb{R}^{m \times n}$ input dataset matrix,
$m =$ number of samples,
$n =$ number of input features,
$\mathbf{w} \in \mathbb{R}^{n}$ (n)−dimensional weight vector,
$b \in \mathbb{R}$ bias term (scalar),
$\hat{\mathbf{y}} \in \mathbb{R}^{m}$ (m)−dimensional output vector

X dataset matrix	w parameters vector
$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1n} \\ x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{mn} \end{bmatrix}$	$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}$

Rows → samples
Columns → features

🎯 Learning Objective

The parameters $\mathbf{w}$ and b are learned by minimizing the **Mean Squared Error (MSE)**:

$$J(\mathbf{w}, b) = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$$

🔁 **Gradient Descent (Vectorized)**. All parameters are updated **simultaneously**:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha\frac{\partial J}{\partial \mathbf{w}}$$

$$b \leftarrow b - \alpha\frac{\partial J}{\partial b}$$

🚀 Key Takeaways

- Multiple Linear Regression expressed in vector form enables efficient computation, clean notation, and scalable learning
- Same gradient descent algorithm — just applied to vectors and matrices 💡

Gradient Descent (Vectorized)

Gradient of the cost function with respect to w:

$$dw = \frac{\partial J}{\partial w} = \frac{2}{m} X^T (\hat{y} - y)$$

- **Why do we use need transpose `X^T @ error`?**
  - `X` shape = `(m, n)`: `m` samples, `n` features
  - `error` = $\hat{y}$ − `y`: shape = `(m,)`
  - We want `dw` shape = `(n,)`: one gradient per weight
  - Transpose reorganizes rows into feature-columns

Regression Evaluation

L1 / L2

Penalty on complexity of model



Regression Error Metrics

- **MSE: Mean squared error** Emphasizes large errors

$$MSE = \frac{1}{\mathrm{m}}\sum(\hat{y} - y)^2$$

- **RMSE: Root mean squared error** Same units as y

$$RMSE = \sqrt{\frac{\sum(\hat{y} - y)^2}{\mathrm{m}}}$$

- **MAE: Mean absolute error** Less sensitive to outliers

$$MAE = \frac{1}{\mathrm{m}}\sum|y - \hat{y}|$$

Prices in QR? MAE and RMSE report error in QR. **Lower is better.**

Underfitting vs Overfitting

❌ **Underfit** → too simple (high bias) → large training & test error

Typical solutions

- Add more relevant features
- Use **polynomial regression**
- Reduce regularization

❌ **Overfit** → too complex (high variance) → low training error, high test error

Typical solutions

- Reduce model complexity e.g., apply **regularization** (Ridge / Lasso)
- Use more training data

✅ **Good fit** → generalize well → low training and test error

→ has appropriate model complexity, proper feature selection and correct regularization strength

Regularization: Motivation & Idea

🚨 Why Regularization?

- Complex models (many features / high-degree polynomials) are prone to **overfitting** ⚠️
- The model fits noise instead of the true pattern

🧠 Core Idea

- Instead of removing features, we: Keep all features, but penalize large weights
- Large weights ⇒ high sensitivity to inputs ⇒ overfitting
- Small weights ⇒ stable predictions ⇒ better generalization ✅

✅ Key Takeaway

Regularization trades a **small increase in bias** for a **large reduction in variance**, improving generalization

Regularization Example

- We predict house prices using: size, number of rooms, age, distance to city center, noise level, and park proximity
- Some are noisy or weak predictors

❌ **No Regularization (Overfitting)**

$$\text{Price} \approx 1200 \cdot \text{Size} + 8000 \cdot \text{Rooms} - 150 \cdot \text{Age} - 30000 \cdot \text{Noise} + 95000 \cdot \text{ParkScore}$$

Issues:

- Very **large weights**
- Small input changes ⇒ **large prediction changes**
- High sensitivity ⇒ **overfitting**

✅ **L2 Regularization (Ridge)**

Penalty: encourages small weights (but rarely zero)

$$\text{Price} \approx 850 \cdot \text{Size} + 6000 \cdot \text{Rooms} - 120 \cdot \text{Age} - 8000 \cdot \text{Noise} + 15000 \cdot \text{ParkScore}$$

Interpretation:

"Use every feature, but don't trust any one too much."

✅ **L1 Regularization (Lasso)**

Penalty: encourages sparsity (some weights → 0)

$$\text{Price} \approx 900 \cdot \text{Size} + 6200 \cdot \text{Rooms} - 130 \cdot \text{Age} + 0 \cdot \text{Noise} + 0 \cdot \text{ParkScore}$$

Interpretation:

"If a feature doesn't help enough, remove it."

✂️ Ridge Regularization (L2)

📐 Cost Function (L1)

$$J(\mathbf{w}) = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2 \; + \; \frac{\lambda}{m}\sum_{j=1}^{n} w_j^2$$

- **First term:** MSE
- **Second term:** model complexity penalty 🧹
- λ regularization strength 🎚️

  Small $\lambda \rightarrow$ weak regularization

  Large $\lambda \rightarrow$ strong regularization

- 📌 **Bias term b is not regularized**

Effect of L2

✅ Keeps **all features**
✅ Shrinks weights **proportionally**
✅ Smooth, stable predictions
❌ No feature elimination

✂️ Lasso Regularization (L1)

📐 **Cost Function (L1)**

$$J(\mathbf{w}) = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2 + \frac{\lambda}{m}\sum_{j=1}^{n}|w_j|$$

- First term: MSE
- Second term: **L1 penalty** (absolute values) ✂️
- λ: regularization strength 🎚️
- Bias term is **not regularized**

Effect of L1

✅ **Some weights become exactly 0**

✅ Noisy / weak features are eliminated

✅ Automatic **feature selection**

✅ Simpler, more interpretable model

✂️ Lasso Regularization (L1)

📐 Cost Function (L1)

$$J(\mathbf{w}) = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2 \; + \; \frac{\lambda}{m}\sum_{j=1}^{n}|w_j|$$

- First term: MSE
- Second term: **L1 penalty** (absolute values) ✂️
- λ: regularization strength 🎚️
- Bias term is **not regularized**

🧠 Key Effect of L1

- During optimization, L1 causes **some weights to become zero**
- Result: **automatic feature selection** ✅

L2 Regularization (Ridge) Gradient

Penalty:

$$\frac{\lambda}{m}\sum_{j=1}^{n} w_j^2$$

Derivative of the penalty:

$$\frac{2\lambda}{m} w_j$$

Total gradient

$$\frac{\partial J}{\partial w_j} = \frac{2}{m}\sum_{i=1}^{m}\left(\hat{y}^{(i)} - y^{(i)}\right)x_j^{(i)} + \frac{2\lambda}{m} w_j$$

Effect Intuition 💡

- L2 penalizes large weights by shrinking them proportionally
- Encourages smooth, stable weight reduction
- Does **not** promote sparsity (weights rarely become exactly zero)

L1 Regularization (Lasso) Gradient

Penalty:

$$\frac{\lambda}{m}\sum_{j=1}^{n} | w_j |$$

Derivative of the penalty

For each weight w_j:

$$\text{sign}(w_j) = \begin{cases} +1, & w_j > 0 \\ -1, & w_j < 0 \\ 0, & w_j = 0 \end{cases}$$

Meaning

- If weight is positive → push downward
- If weight is negative → push upward
- If weight is zero → keep at zero

Total (sub)gradient

$$\frac{\partial J}{\partial w_j} = \frac{2}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})x_j^{(i)} + \frac{\lambda}{m}\,\text{sign}(w_j)$$

Effect Intuition 💡

- L1 term adds a **constant pull toward zero**
- Many weights become exactly zero
- Zero weights = automatic **feature selection**

🚀 L1 vs L2 Comparison

➡️ Weights are **continuously shrunk** toward zero 🔽

- **Ridge (L2)**: smooth shrinking, keeps all features
- **Lasso (L1)**: some weights become zero ✂️

→ This often improves generalization when features are many or noisy

Aspect	L2 (Ridge)	L1 (Lasso)
Keeps all features	✅	❌
Shrinks large weights	✅	✅
Produces zero weights	❌	✅
Feature selection	❌	✅
Model stability	High	Medium

Takeaways

- 🎚️ Regularization controls model complexity
- ✅ Essential for high-dimensional models